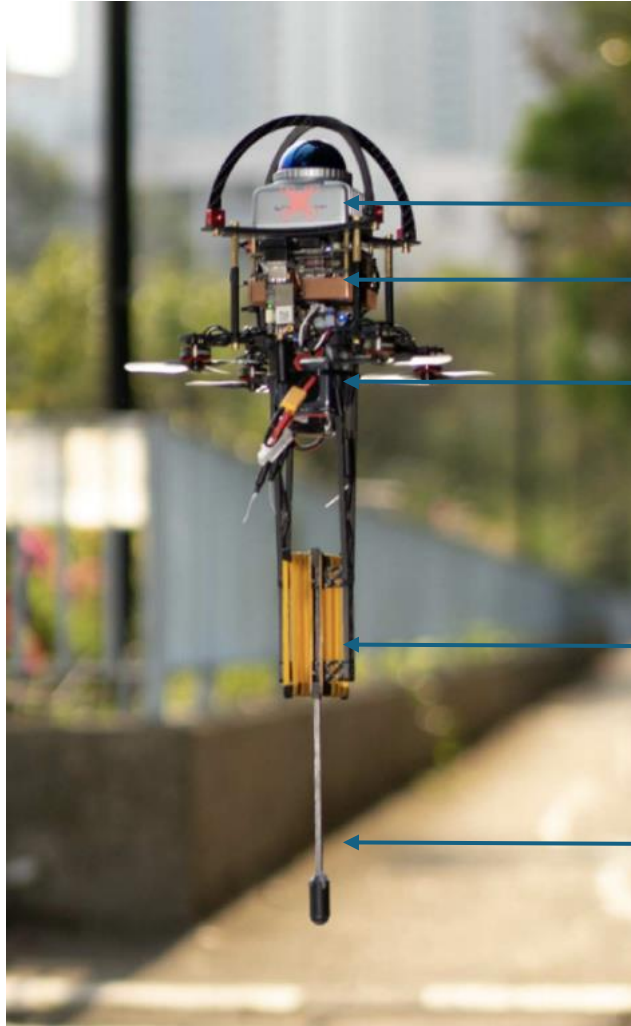




→ Livox mid360 LiDAR for perception

→ Onboard computer

→ Quadcopter

→ Elastic bands for hopping

→ Leg

Perception-Informed Autonomous Navigation for a Monopedal Hopping Robot

Zwe Min Htet Aung (Larry)

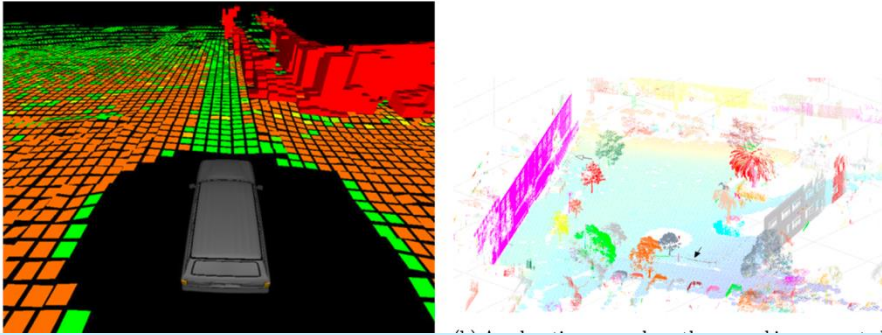
S. Li et al., "A High-Payload Robotic Hopper Powered by Bidirectional Thrusters," in IEEE Transactions on Robotics, vol. 41, pp. 5307-5326, 2025, doi: 10.1109/TRO.2025.3600127.

Master thesis

Progress update

Hopcopter uses LiDAR to navigate around obstacles

Perception and Mapping

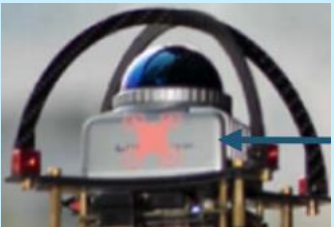


Visualization of terrain roughness, Neuhaus et al.

Elevation map, Douillard et al.

Perception tools can either be a **LiDAR**, **camera**, or both and provide crucial information for environment representation and planning

Current implementation

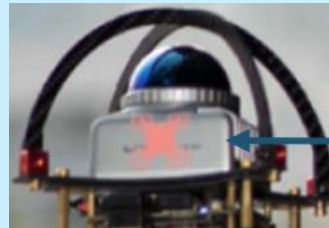


LiDAR

Pose estimation

Pose estimation is the process of determining the position and orientation of a body or sensor with respect to a reference frame. This is commonly achieved through **external motion capture**, **LiDAR-inertial odometry (LIO)**, or visual-inertial odometry (**VIO**).

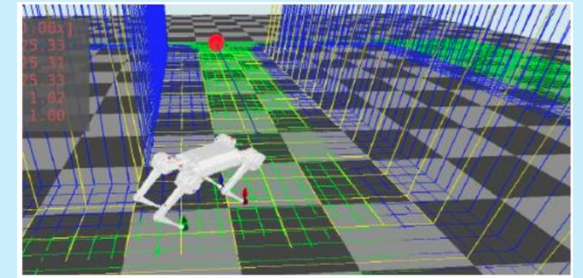
Current implementation



LiDAR inertial odometry

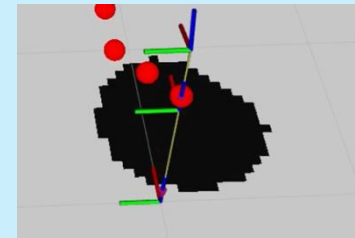
Planning

Motion planning is the problem a collision-free path through a robot's configuration space



A quadruped navigating a hallway, Dudzik et al.

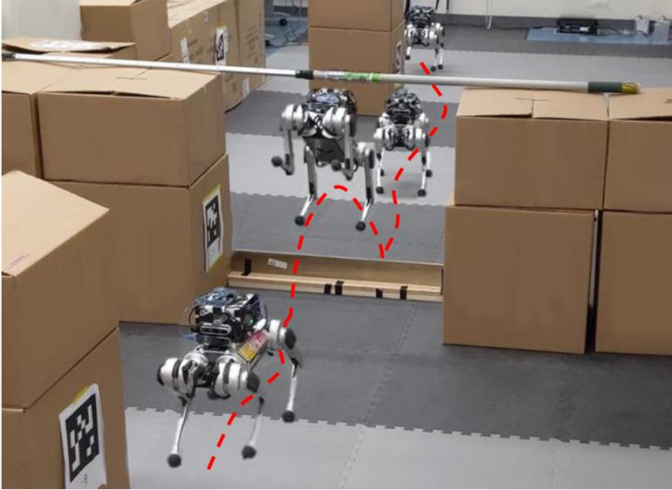
Current implementation



RRT*

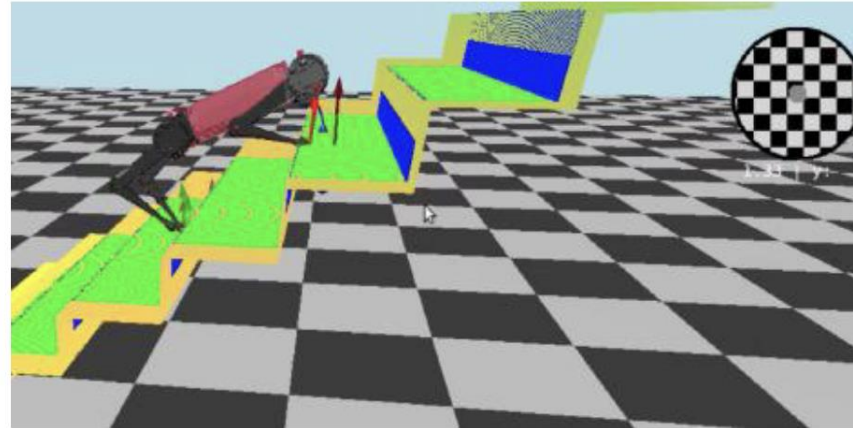
Motion planning algorithms in quadrupeds combine global and local planning

Gilroy et al.



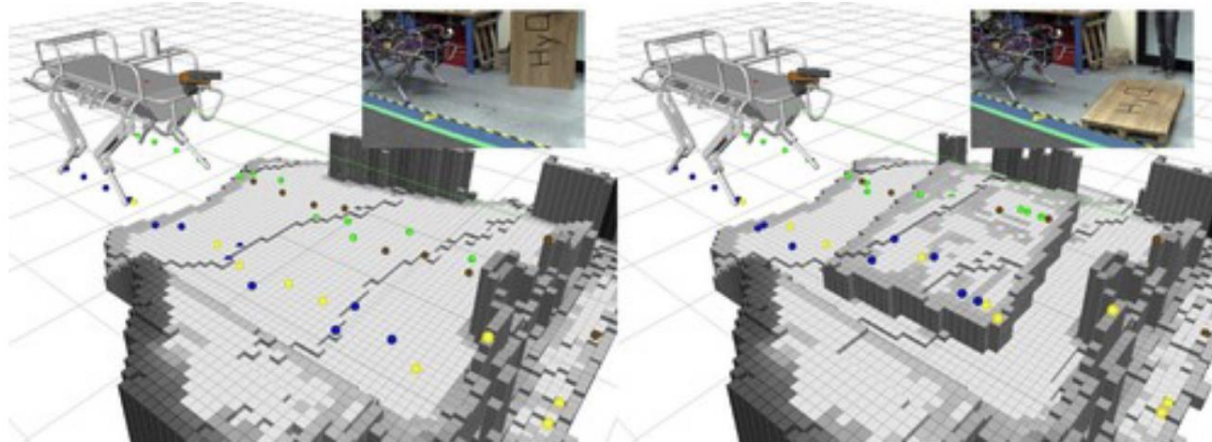
S. Gilroy *et al.*,
“Autonomous
Navigation for
Quadrupedal Robots
with Optimized Jump
through Constrained
Obstacles,” July 01,
2021, *arXiv*:
[arXiv:2107.00773](https://arxiv.org/abs/2107.00773). doi
[10.48550/arXiv.2107.00773](https://doi.org/10.48550/arXiv.2107.00773).

Dudzik et al.



T. Dudzik *et al.*, “Robust Autonomous
Navigation of a Small-Scale Quadruped
Robot in Real-World Environments,” in
*2020 IEEE/RSJ International Conference
on Intelligent Robots and Systems (IROS)*,
Las Vegas, NV, USA: IEEE, Oct. 2020, pp.
3664–3671.

Winkler et al.



A. W. Winkler, C. Mastalli, I. Havoutis, M. Focchi, D. G. Caldwell,
and C. Semini, “Planning and Execution of Dynamic Whole-Body
Locomotion for a Hydraulic Quadruped on Challenging Terrain,”
in *2015 IEEE International Conference on Robotics and
Automation (ICRA)*, May 2015, pp. 5148–5154.

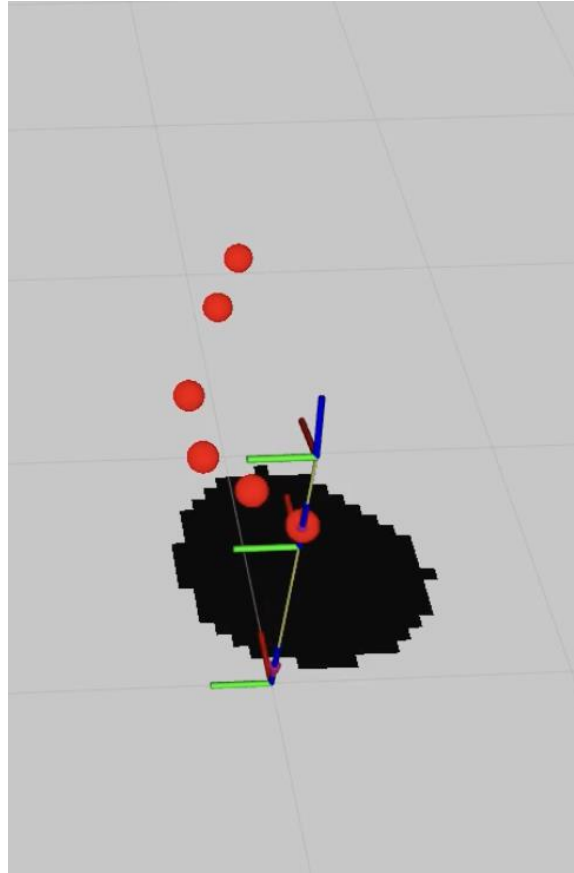
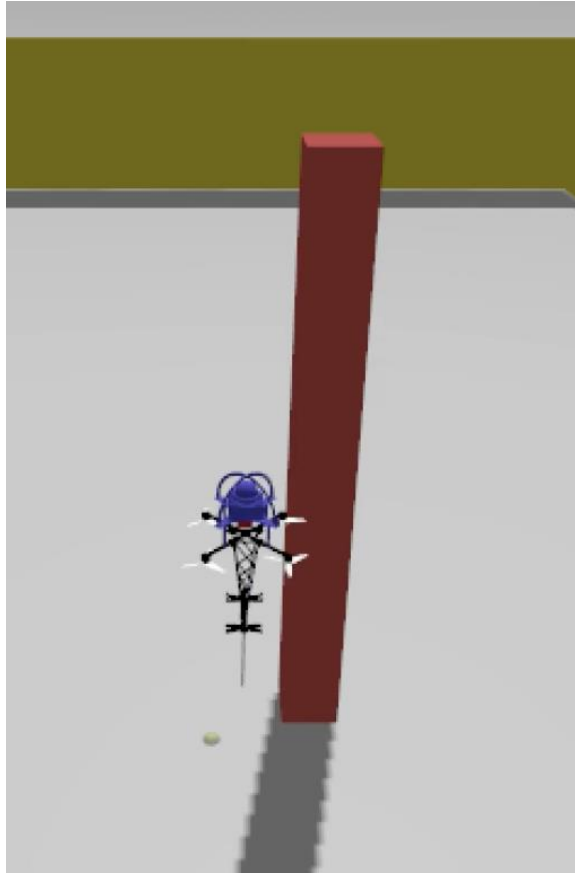
Global planner will lay out the initial path or provide a general direction



Figure 4.3: Scene photograph and RVIZ representation of Hopcopter following the planned path.

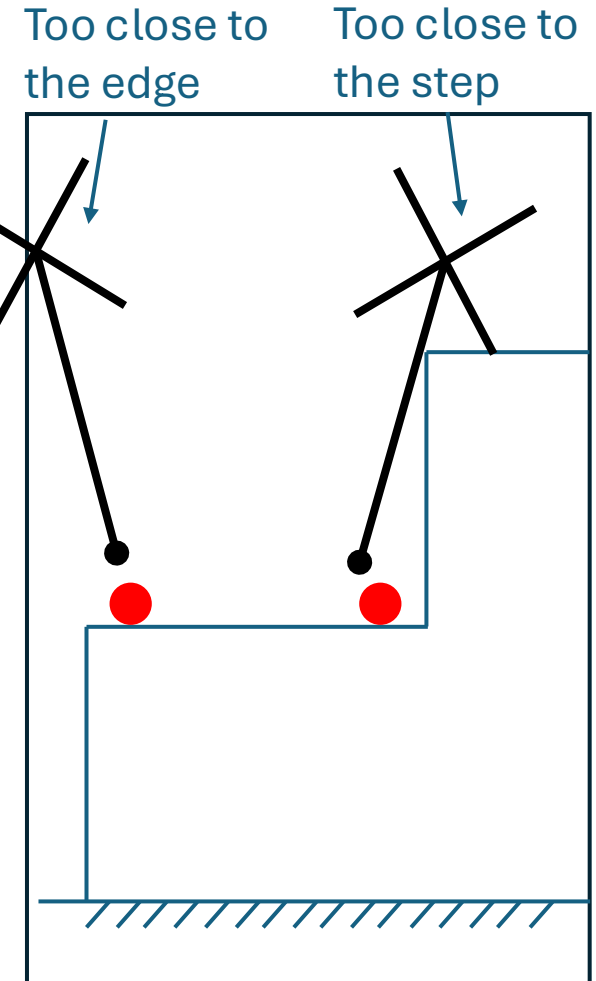
Running the global planner alone has limitations; Hopcopter needs a local planner to increase its traversal capabilities

Capabilities



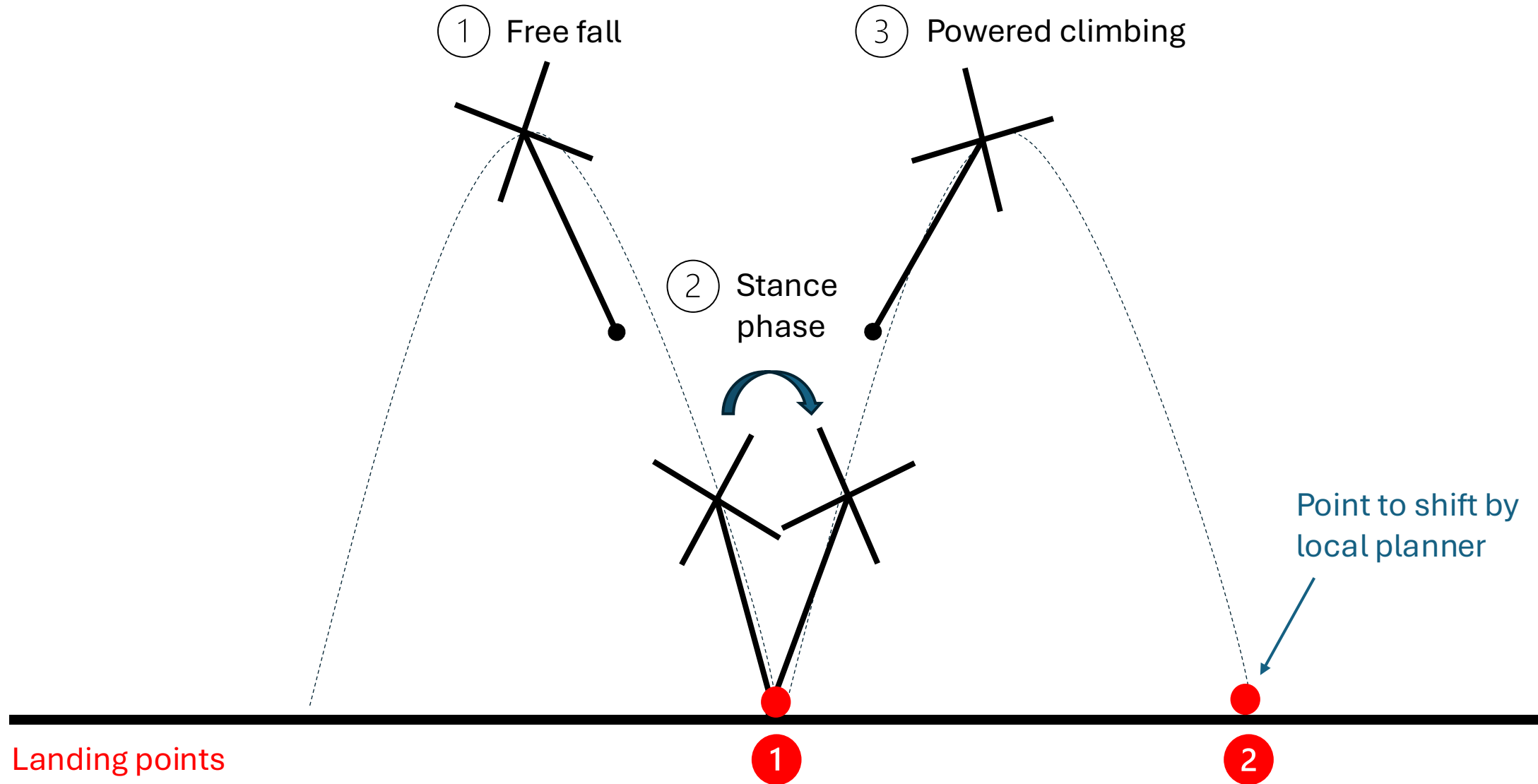
- Can create a path that avoids obstacles

Limitations



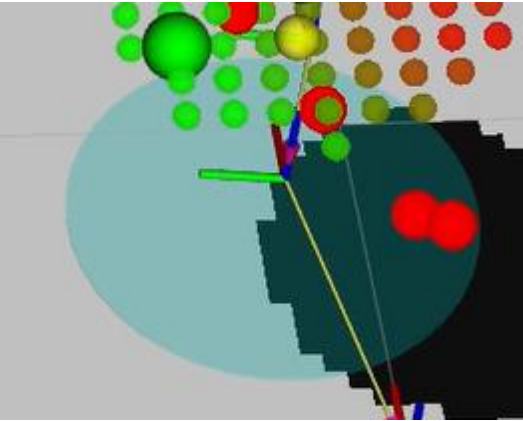
1. Unable to handle sudden appearances of obstacles
2. Unable to handle smaller stepping regions or stairs

Jumping dynamics of Hopcopter require local planner to adjust Point 2



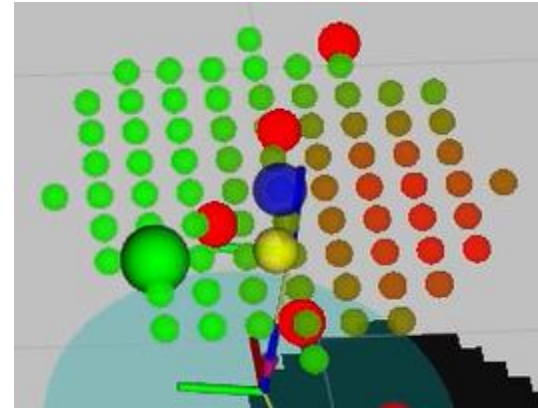
Current implementation of the local planner discretizes the region and scores landing points based on two criteria

1



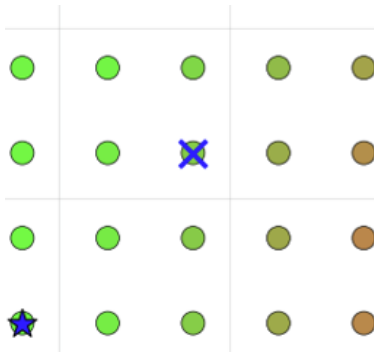
Mark a circular region of radius 0.5 m around the waypoint of interest (Point 2)

2



Discretize the region into candidate landing points

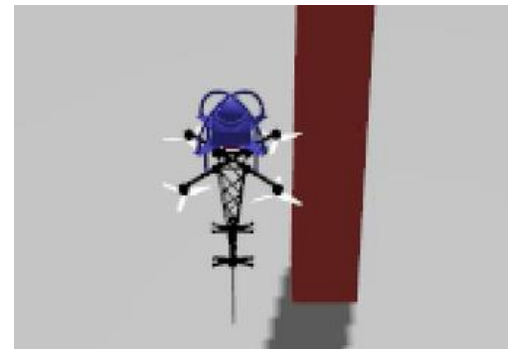
3



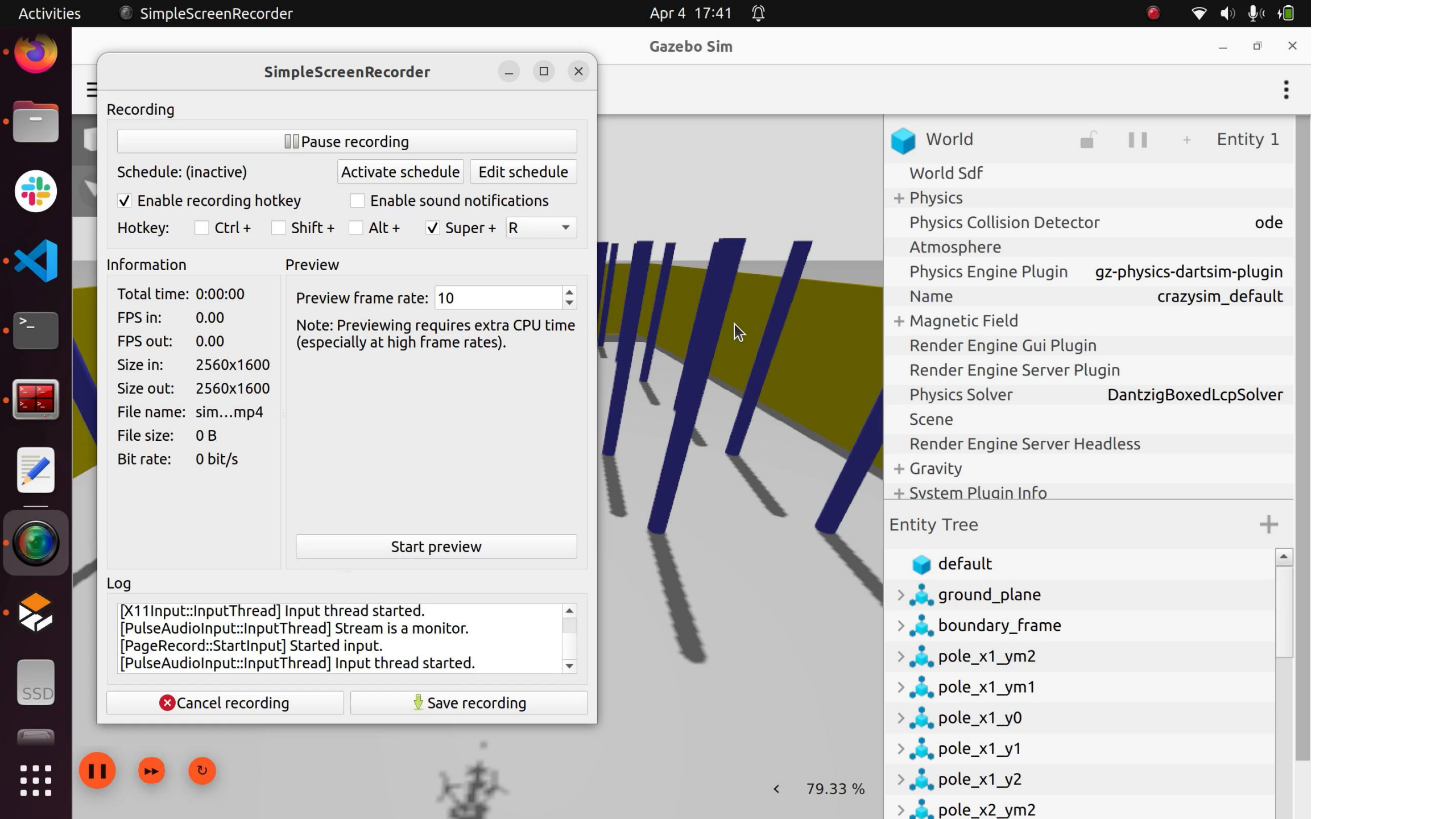
Score the candidate landing points based on:

1. **Proximity to nearby obstacles**
2. **Distance from takeoff point (Point 1)**

4



Queue the selected landing point as the next waypoint to follow



SimpleScreenRecorder

Recording

Pause recording

Schedule: (inactive)

Activate schedule

Edit schedule

☒ Enable recording hotkey☐ Enable sound notifications

Hotkey:

☐ Ctrl +☐ Shift +☐ Alt +☒ Super +

R

Information

Total time: 0:00:00

FPS in: 0.00

FPS out: 0.00

Size in: 2560x1600

Size out: 2560x1600

File name: sim...mp4

File size: 0 B

Bit rate: 0 bit/s

Preview

Preview frame rate: 10

Note: Previewing requires extra CPU time (especially at high frame rates).

Start preview

Log

[X11Input::InputThread] Input thread started.
[PulseAudioInput::InputThread] Stream is a monitor.
[PageRecord::StartInput] Started input.
[PulseAudioInput::InputThread] Input thread started.

Cancel recording

Save recording

World

World Sdf

+ Physics

Physics Collision Detector

ode

Atmosphere

Physics Engine Plugin

gz-physics-dartsim-plugin

Name

crazysim_default

+ Magnetic Field

Render Engine Gui Plugin

Render Engine Server Plugin

Physics Solver

DantzigBoxedLcpSolver

Scene

Render Engine Server Headless

+ Gravity

+ System Plugin Info

Entity Tree

default

> ground_plane

> boundary_frame

> pole_x1_ym2

> pole_x1_ym1

> pole_x1_y0

> pole_x1_y1

> pole_x1_y2

> pole_x2_ym2

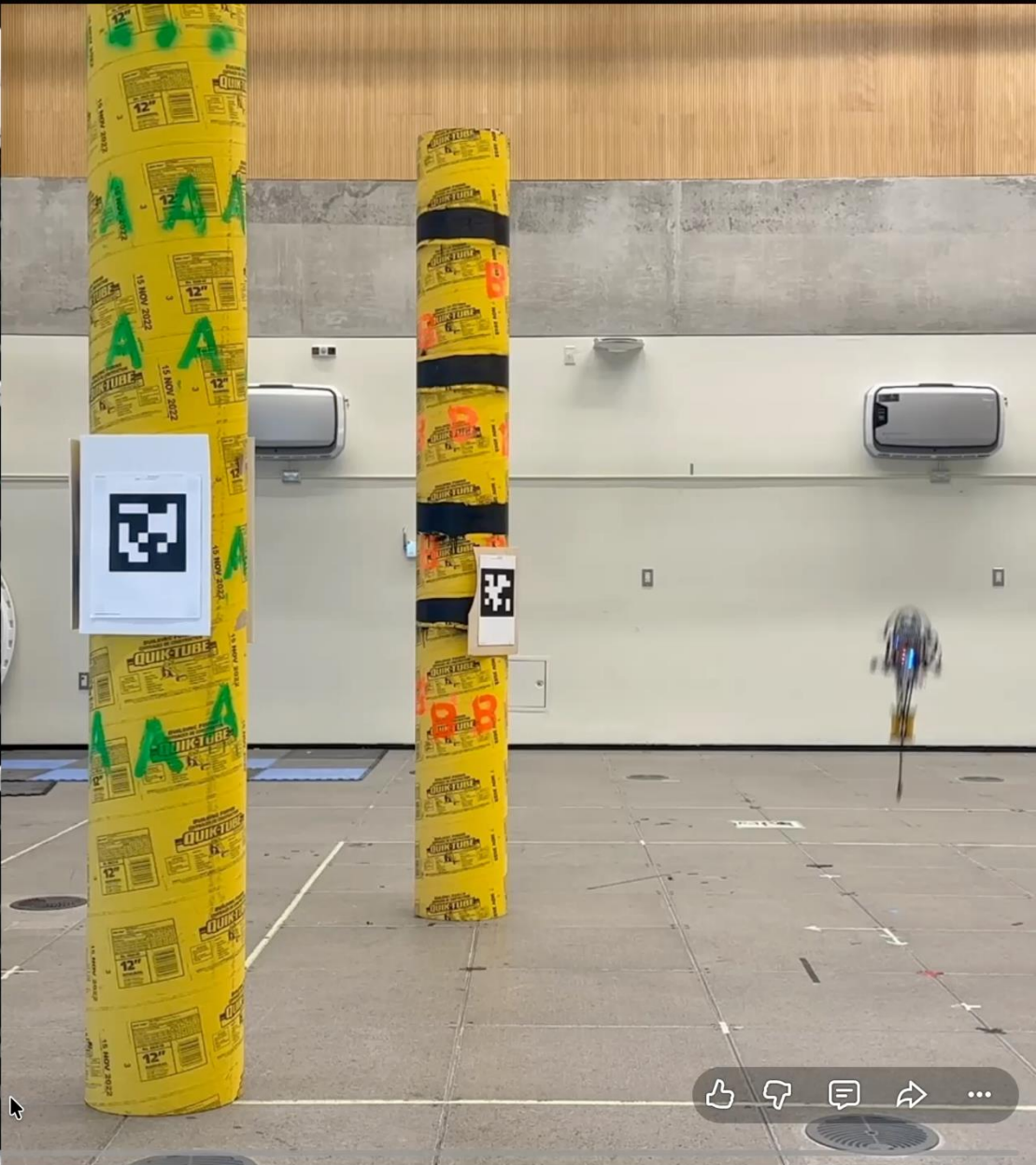
Fastlio, 1 goal (split video)

```
Apr 23 14:13
/home/hopcopter/ws_fastlio/install/fast_llo/share/fast_llo/rviz/fastlio.rviz* - RViz

hopcopter@hopcopter-ROUTER-2L: ~
hopcopter@hopcopter-ROUTER-2L: ~$ ros2 launch fastlio fastlio.launch
pattern mode, handle: 2936121536, set_b
lidar attitude, ip: 192.168.1.175
nge work mode, handle: 2936121536
le Livox Lidar imu, ip: 192.168.1.175
74.372424839] [livox_lidar_publisher]: l
74.373906859] [livox_lidar_publisher]: l

hopcopter@hopcopter-ROUTER-2L: ~$ ros2 launch fastlio fastlio.launch
publisher: beginning loop
publishing #1: geometry_msgs.msg.PoseSta
ped(header=std_msgs.msg.Header(stamp=bu
ltin_interfaces.msg.Time(sec=0, nanosec
0), frame_id='world'), pose=geometry_ms
s.msg.Pose(position=geometry_msgs.msg.P
int(x=2.0, y=0.0, z=0.3), orientation=g
ometry_msgs.msg.Quaternion(x=0.0, y=0.0
z=0.0, w=1.0))
hopcopter@hopcopter-ROUTER-2L: ~$ ros2 to
ic pub --once /goal_pose geometry_msgs/
msg/PoseStamped "{header: {frame_id: 'wo
ld'}, pose: {position: {x: 2.0, y: 0.0,
z: 0.3}, orientation: {x: 0.0, y: 0.0,
: 0.0, w: 1.0}}}"

hopcopter@hopcopter-ROUTER-2L: ~$ ros2 launch fastlio fastlio.launch
[fastlio_mapping-1] IMU Initial Done
[fastlio_mapping-1] [WARN] [1776967883.832326693] [laser_m
hls scan]
[fastlio_mapping-1]
[rviz2-2] [INFO] [1776967883.841079675] [rviz2]: Stereo is
[rviz2-2] [INFO] [1776967883.841169638] [rviz2]: OpenGL ve
[rviz2-2] [INFO] [1776967883.859780501] [rviz2]: Stereo is
[fastlio_mapping-1] [INFO] [1776967883.883028234] [laser_m
ap kdtee
[rviz2-2] [INFO] [1776967894.635579116] [rviz2]: Trying to
x 200 using 1 swatches
[rviz2-2] [ERROR] [1776967894.668182708] [rviz2]: Vertex P
ed_8bit_image.vert Fragment Program:rviz/gls1120/indexed_8
result :
[rviz2-2] active samplers with a different type refer to t
```



Waypoint following is unstable because jumping model needs tuning

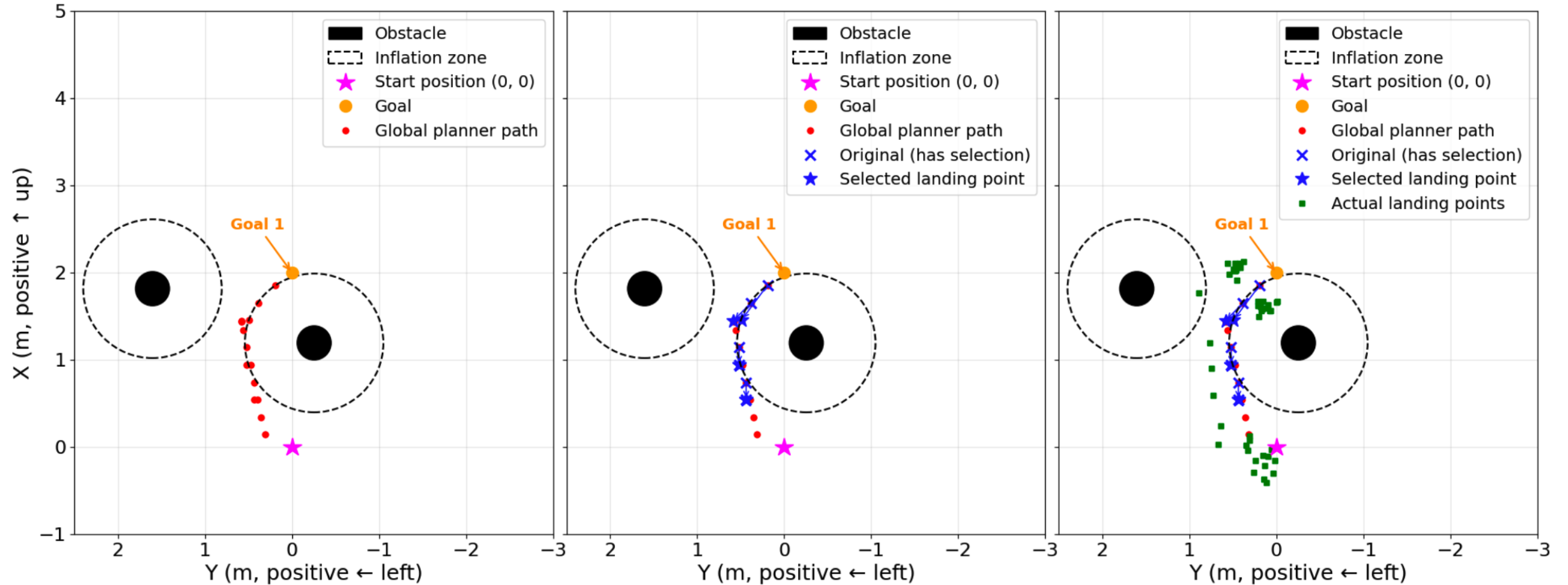


Figure 4.8: Path following results of Hopcopter using onboard state estimation [10].

Master thesis

Progress update

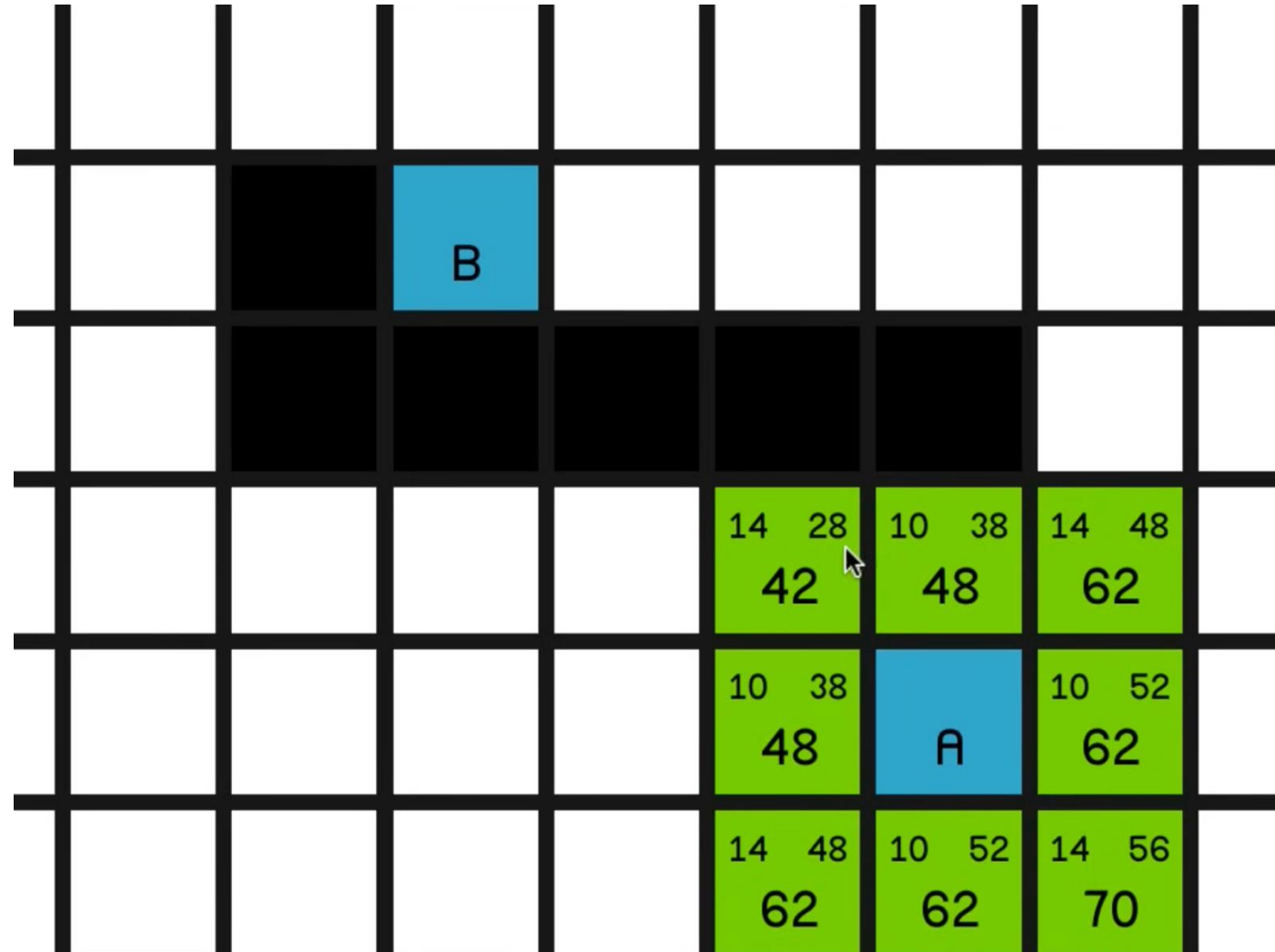
Search loop for Hopping A* is the same as classic A*

CLASSIC A*

```
A*(start, goal)
  open ← {start}, g[start] ← 0
  while open not empty
    current ← pop lowest f = g + h
    if current = goal → return path

    for each n in Neighbors(current)
      c ← g[current] + EdgeCost(current, n)
      if c < g[n]
        g[n] ← c; parent[n] ← current
        push n with f = c + h(n)

  return failure
```



Hopping A* is different in two areas

CLASSIC A*

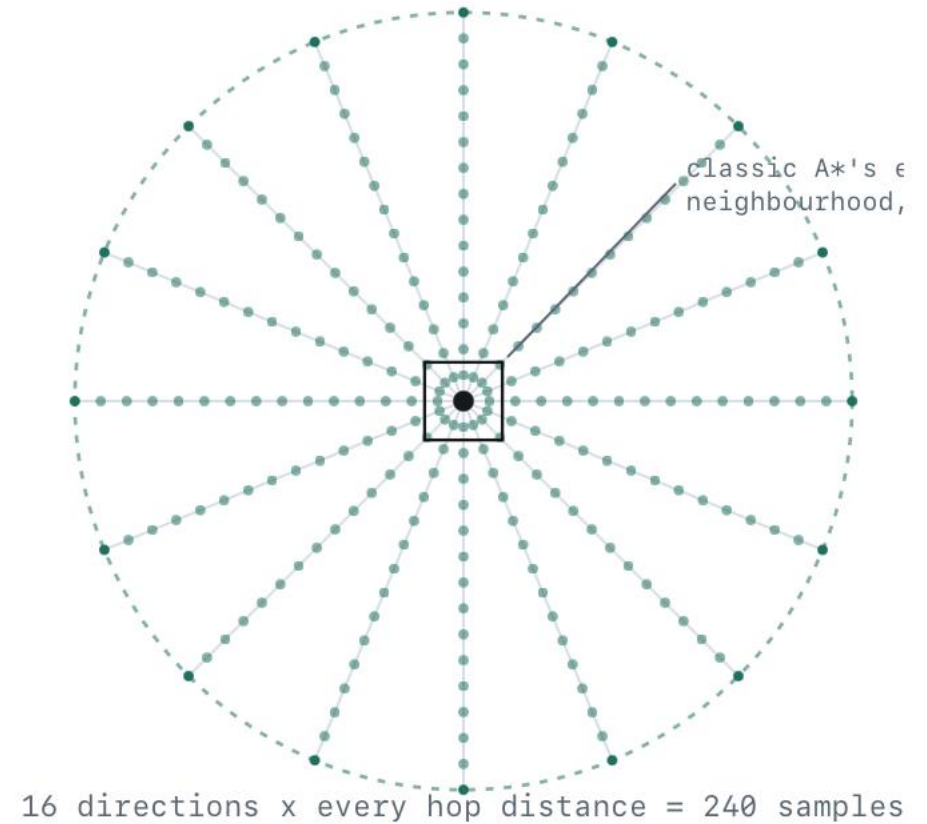
```
A*(start, goal)
  open ← {start}, g[start] ← 0
  while open not empty
    current ← pop lowest f = g + h
    if current = goal → return path

    for each n in Neighbors(current)
      c ← g[current] + EdgeCost(current, n)
      if c < g[n]
        g[n] ← c; parent[n] ← current
        push n with f = c + h(n)

  return failure
```

**Deciding which
nodes count as
“neighbors”**

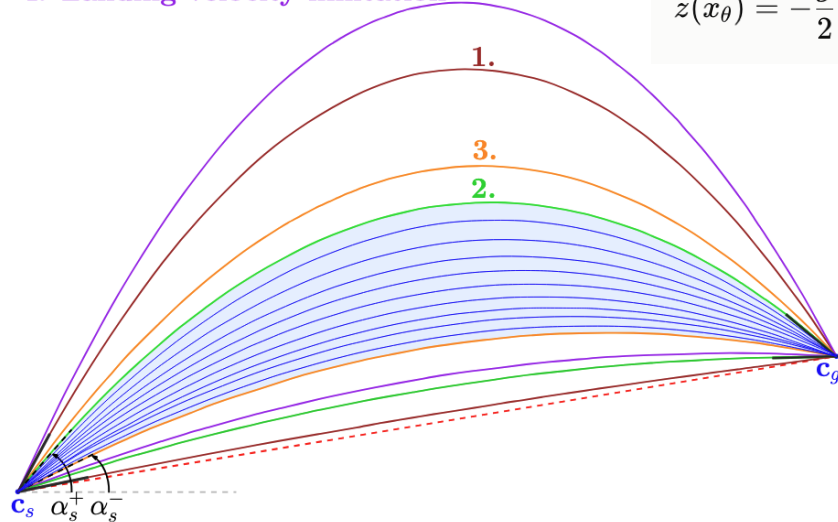
**Cost of a single
move**



Neighbour selection has 3 requirements

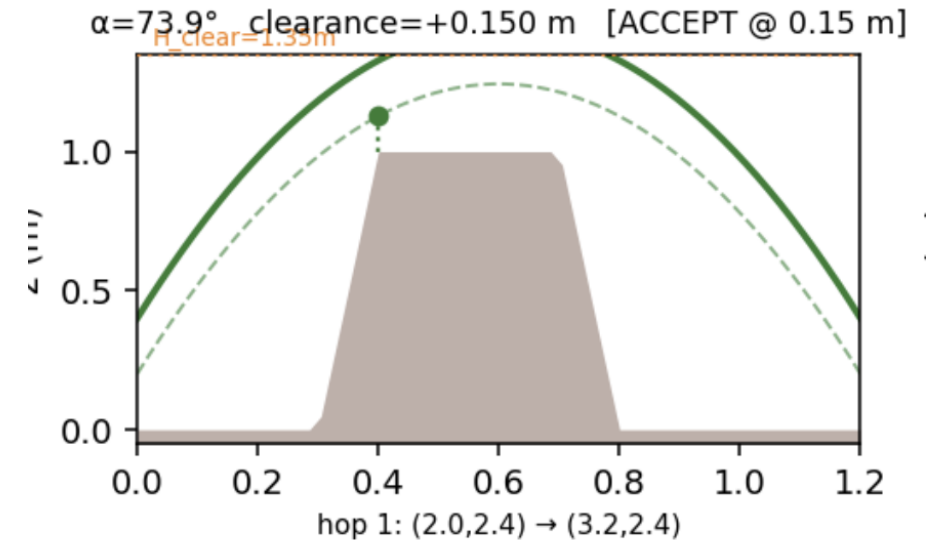
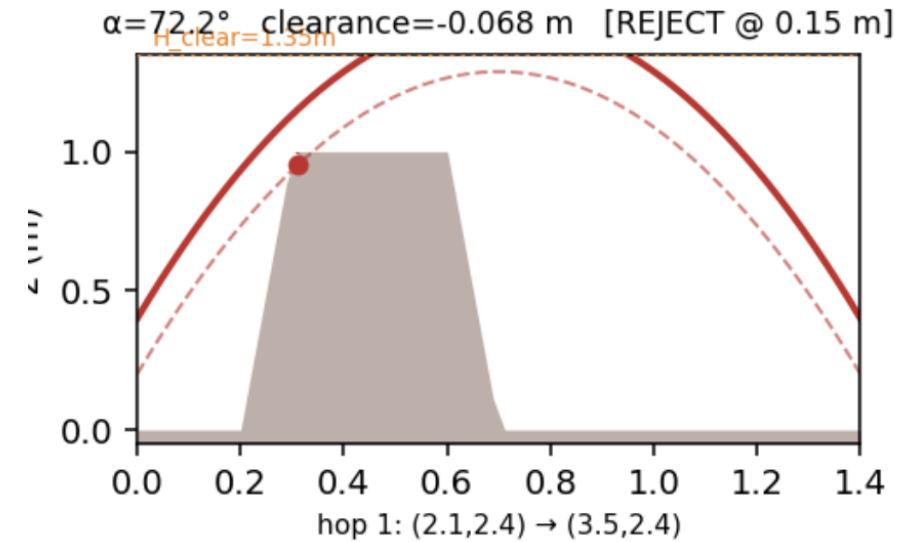
1. Landing point must not be an obstacle (Classic A*)
2. Landing point is **reachable by the takeoff angles** (α) the leg can produce (Campana et al.)
3. At least one parabola that is produced within α range **must not collide** with any obstacle/terrain (Campana et al.)

1. Takeoff from initial cone
2. Landing in final cone
3. Takeoff velocity limitation
4. Landing velocity limitation



$$z(x_\theta) = -\frac{g}{2} \frac{(x_\theta - x_{\theta_s})^2}{\dot{x}_{\theta_s}^2} + \frac{\dot{z}_s}{\dot{x}_{\theta_s}} (x_\theta - x_{\theta_s}) + z_s$$

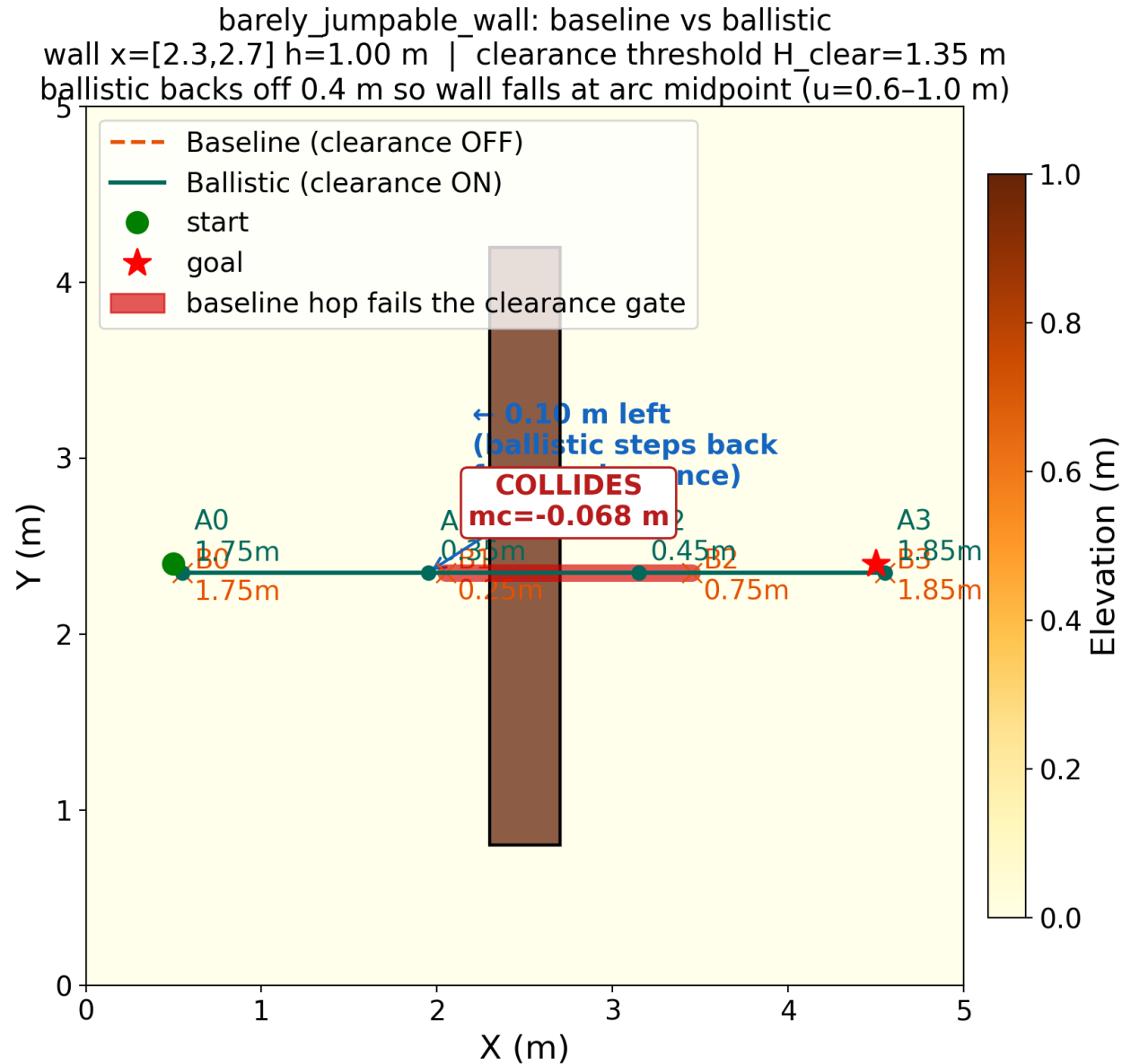
Example



Edge cost also includes

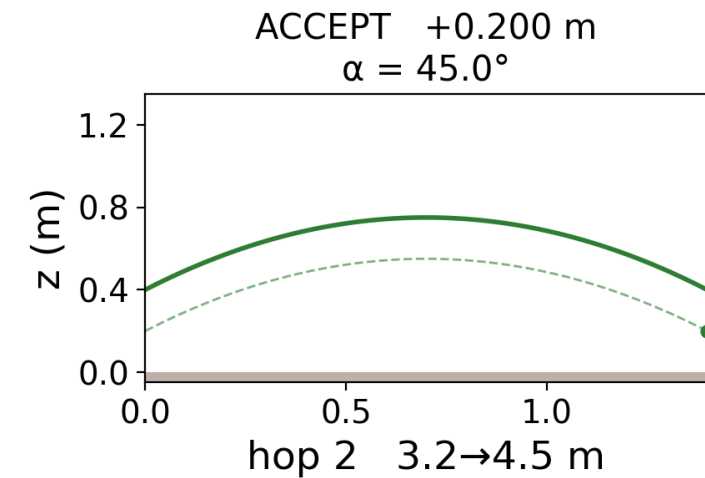
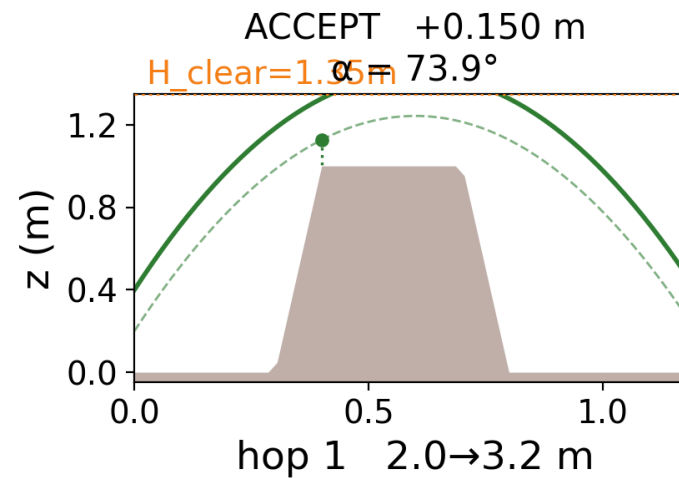
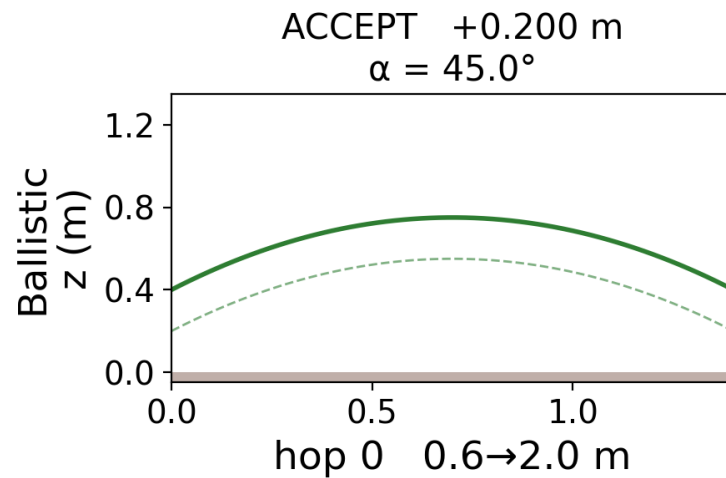
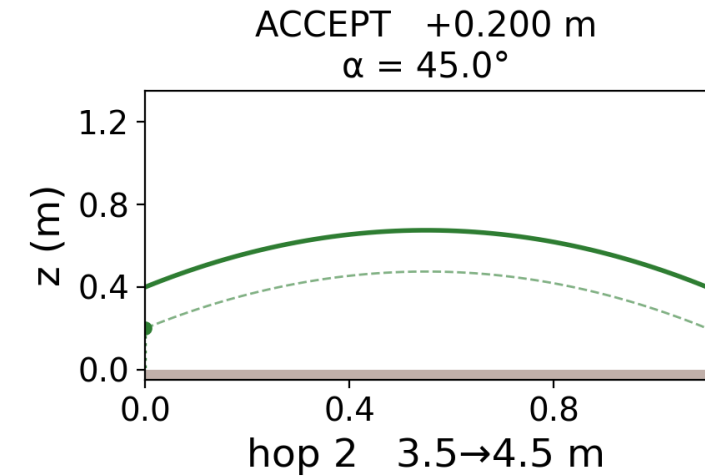
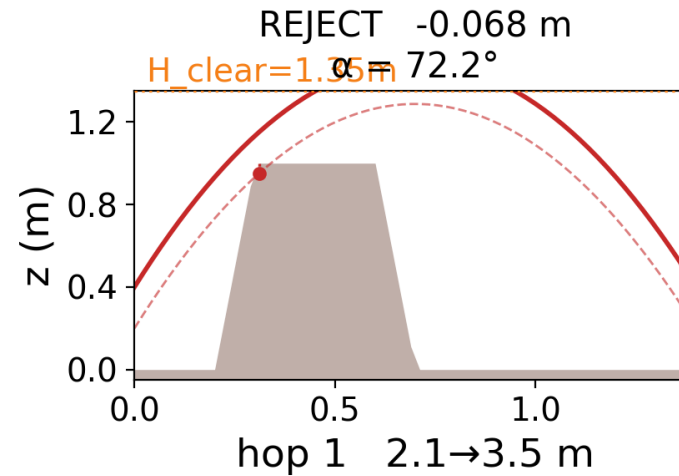
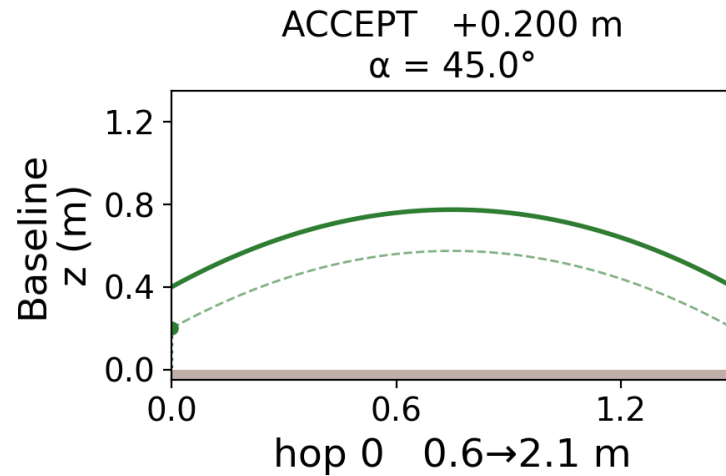
1. Distance from current node to neighbor node
(Classic A^*)
2. Elevation penalty (not demonstrated yet)
3. Per-hop cost

Clearance checking results in a parabola that starts off slightly behind

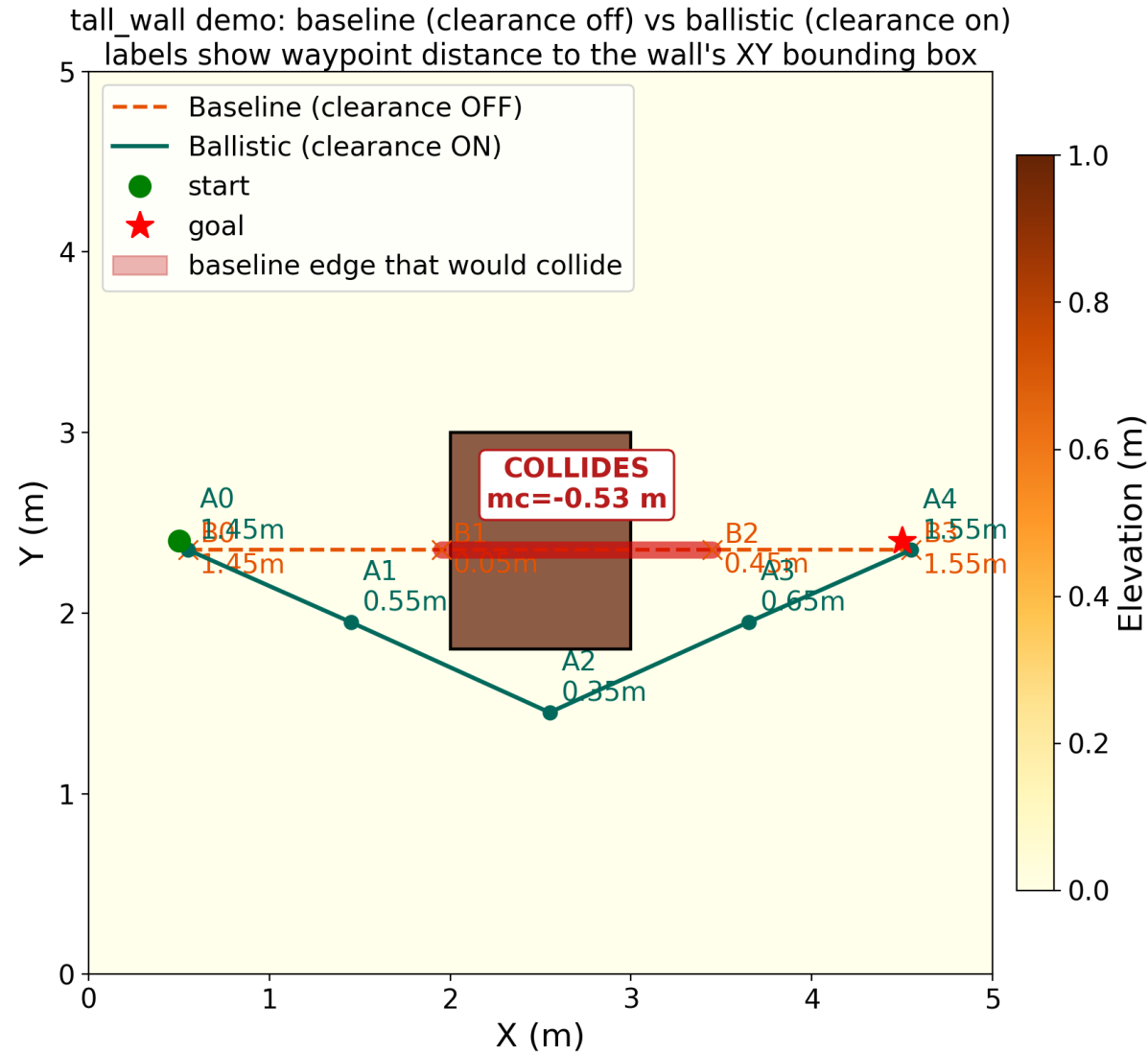


Clearance checking results in a parabola that starts off slightly behind

Per-hop side view — baseline (top) vs ballistic (bottom)
orange dotted = H_{clear} 1.35 m · red = below the 0.15 m gate

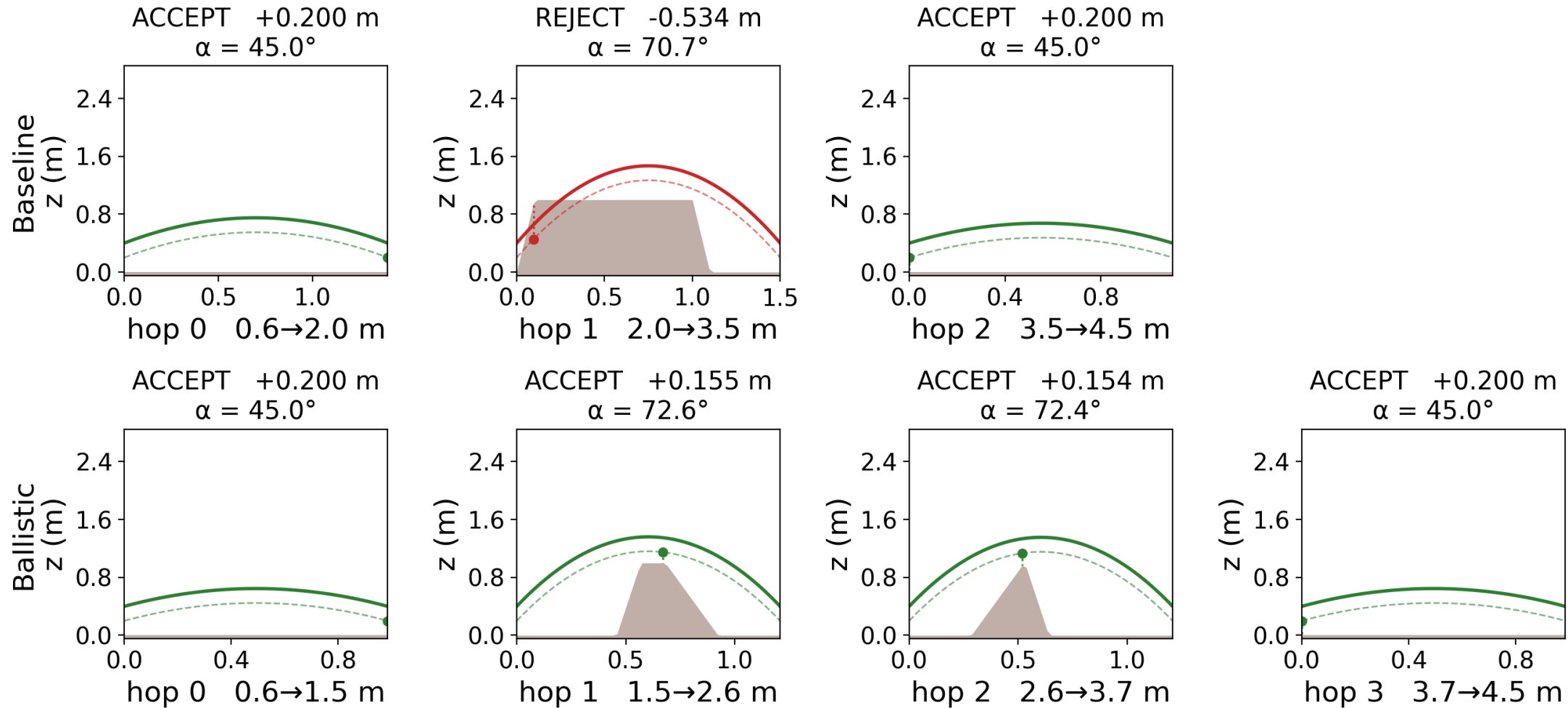


For a wide wall, the algorithm will plan around it

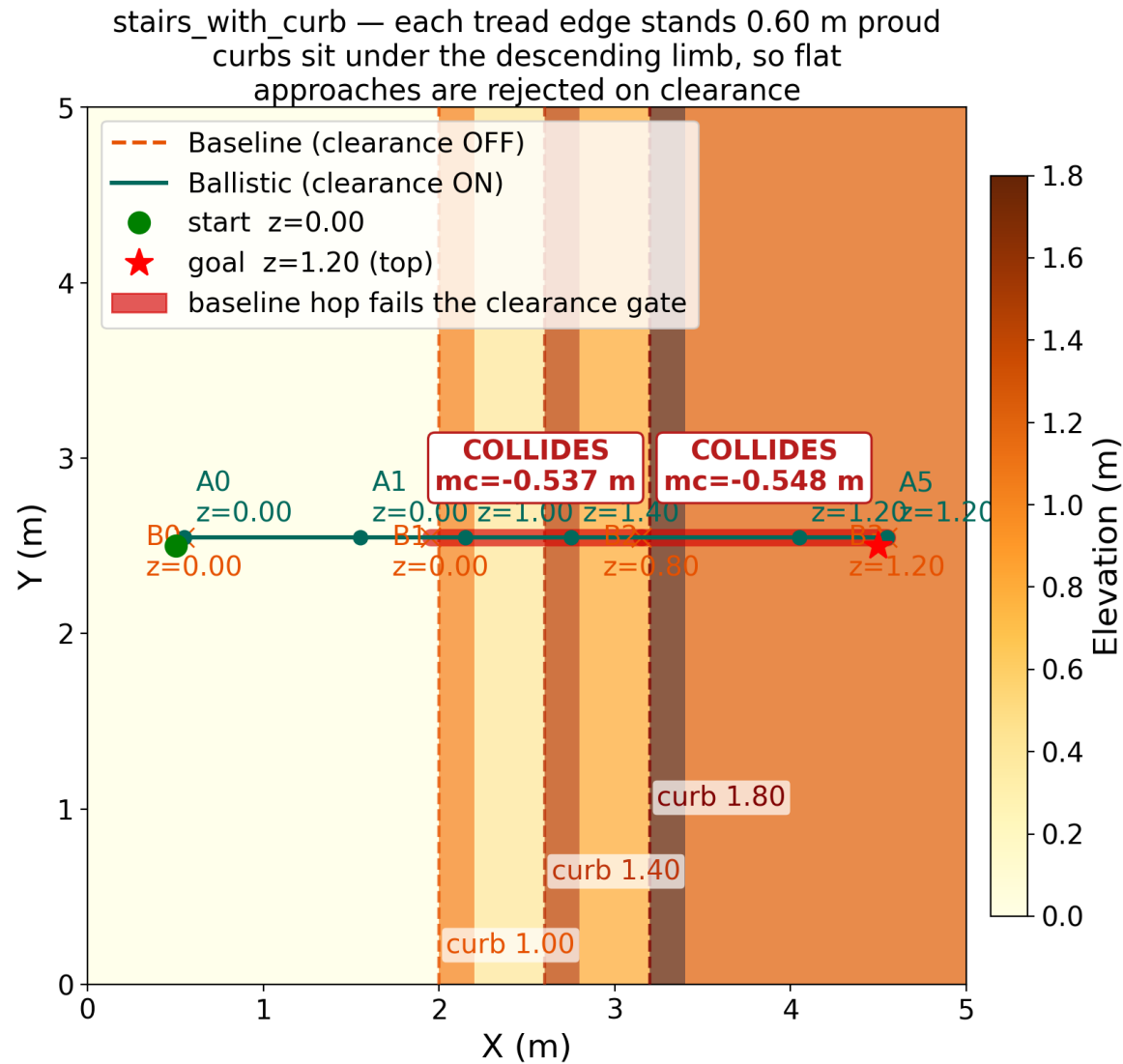


For a non-jumpable wall, the algorithm will plan around it

Per-hop side view — baseline (top) vs ballistic (bottom)
red = below the 0.15 m clearance gate · both paths judged by the same criterion



Demo on stairs with a curb



Demo on stairs with a curb

Per-hop side view — baseline (top) vs ballistic (bottom)
curbs at 1.00 / 1.40 / 1.80 m · red = below the 0.15 m gate

